

```

1  """
2  -----
3  File: GedcomNameOverlay.py
4
5  Summary: The "Label-Maker" for the Census-First Architecture.
6           This script takes one or more auxiliary GEDCOM files, extracts
7           the demographic fingerprints of the families within, and uses
8           them to find and "paint" names onto the nameless, mathematically
9           proven family structures in our Master Relational Vault.
10
11  Design:
12  - Step 1: Ingest all auxiliary GEDCOMs (Personal, WikiTree, etc.) into a
13            temporary database table.
14  - Step 2: For each GEDCOM family, create the same 10-variable demographic
15            hash we use in the Time Machine (Head/Spouse Ages, BPLs, FBPLs, MBPLs).
16  - Step 3: Use that hash to perform a direct, lightning-fast SQL lookup
17            against the MasterVault_Relational.db.
18  - Step 4: If a unique match is found, update the "Future Bosselstink"
19            placeholders in the 'individuals' table with the correct
20            historical names from the GEDCOM.
21  -----
22  """
23
24  import os
25  import sys
26  import duckdb
27
28  # Add the 'python' directory and project root to sys.path so we can import properly
29  script_dir = os.path.dirname(os.path.abspath(__file__))
30  project_root = os.path.abspath(os.path.join(script_dir, '..'))
31  for p in [os.path.join(project_root, 'python'), project_root]:
32      if p not in sys.path:
33          sys.path.append(p)
34
35  from utils import gen_logging
36
37  # =====
38  # CONFIGURATION
39  # =====
40  if os.name == 'nt':
41      BASE_DATA_DIR = r"D:\Data\Genealogy_Data"
42  else:
43      BASE_DATA_DIR = os.path.expanduser("~/Genealogy_Data")
44
45  # The database we are reading from (our ground truth)
46  MASTER_DB = os.path.join(BASE_DATA_DIR, "MasterVault_Relational.db")
47
48  # The database we are writing the name changes to.
49  # DECISION: We create a NEW database for the overlay. This keeps our raw
50  # relational vault immutable. The final export will join this overlay
51  # with the master vault to get the complete picture.
52  NAMED_DB = os.path.join(BASE_DATA_DIR, "MasterVault_Named.db")
53
54  # A directory where you can drop all your GEDCOM files (personal, WikiTree, etc.)
55  GEDCOM_INPUT_DIR = os.path.join(project_root, "gedcom_sources")
56
57
58  def apply_gedcom_names(logger):
59      """
60      Finds nameless families in the Master Vault that match the demographic
61      fingerprint of families in the auxiliary GEDCOM files and applies the
62      correct historical names.
63      """
64      logger.info("Initializing DuckDB for GEDCOM Name Overlay...")
65      con = duckdb.connect()
66      con.execute("PRAGMA memory_limit='90GB'")

```

```

67 con.execute("INSTALL sqlite; LOAD sqlite;")
68
69 # -----
70 # Step 1: Ingest all GEDCOM files
71 # -----
72 logger.info(f"Scanning for GEDCOM files in: {GEDCOM_INPUT_DIR}")
73 os.makedirs(GEDCOM_INPUT_DIR, exist_ok=True)
74
75 # This is where we will add the logic to parse every .ged file in the
76 # directory and load them into a temporary 'gedcom_families' table.
77 # For now, we will just print a placeholder message.
78
79 logger.info("--> [DRAFT] GEDCOM Ingestion Logic will go here.")
80 logger.info("--> [DRAFT] It will parse all .ged files and create a temp table.")
81
82 # -----
83 # Step 2: Create Demographic Hashes
84 # -----
85 logger.info("Creating demographic fingerprints for all GEDCOM families...")
86
87 # This is where we will write the SQL to create the 10-variable hash
88 # for every family found in the GEDCOM files.
89
90 logger.info("--> [DRAFT] Demographic Hash creation logic will go here.")
91
92 # -----
93 # Step 3: Perform the SQL Lookup
94 # -----
95 logger.info("Searching for nameless 'Bosselstink' families in the Master Vault...")
96
97 # This is where we will JOIN our GEDCOM hashes against the hh_features
98 # we created in the Time Machine to find perfect matches.
99
100 logger.info("--> [DRAFT] SQL Join logic to find matches will go here.")
101
102 # -----
103 # Step 4: Update the Names
104 # -----
105 logger.info("Applying historical names to the matched census records...")
106
107 # This is where we will create the new NAMED_DB and run an UPDATE
108 # command to replace "Future Bosselstink" with the correct names.
109
110 logger.info("--> [DRAFT] UPDATE logic to 'paint' names will go here.")
111
112 logger.info("\nGEDCOM Name Overlay process draft complete!")
113 logger.info("This script is ready for us to build out the real logic.")
114
115
116 if __name__ == "__main__":
117     main_logger = gen_logging.setup_logging(logger_name="NAME_OVERLAY")
118     apply_gedcom_names(main_logger)

```